

Candidate-Set Objectives for Robotic Ball Interception: Benefits and Limitations under Speed Constraints

Abstract—Robotic tennis hitting needs swings that stay valid under ball-prediction errors, yet point targets compress the solution into a single, prediction-sensitive point. Starting from an MPC-iLQR point-target terminal, we study two switches of the objective: a spatial corridor (hinge running cost along the predicted ball trajectory, racket radius as half-width) and a candidate-window terminal replacing the point by a differentiable softmin over candidate states along that trajectory. On a dual-arm RM-65B in simulation (123k episodes, same-seed pairing), a four-way ablation, a perturbation grid, and a speed-limit pairing test them. The candidate window carries nearly all the nominal gain over the point target (+17.4 pp at 9 m/s, +32.7 pp at 12 m/s); once it is active the corridor adds no detectable gain. Under the nominal 1.0 m/s TCP limit the soft-aggregation tiers fall below the point target (−11.9 to −12.7 pp) while the corridor-only tier still gains. Each layer’s regime of benefit and failure is delimited; the cost structure should follow the speed budget.

Index Terms—model predictive control, iterative linear-quadratic regulator, trajectory optimization, ball hitting, uncertainty-aware planning

I. INTRODUCTION

Robot manipulation is increasingly expected to handle *dynamic* tasks in which the target is in motion. Hitting a ball is a canonical instance: the end-effector must meet a fast, small target at a single instant, with a prescribed face pose and speed, and the swing must be committed in advance of contact on the strength of a ball prediction. Any plan therefore rests on a prediction, and prediction error is unavoidable (sensing noise, model mismatch). Its cost is asymmetric: a 0.2 m hit-point offset alone lowers the full-system hit rate by 21.5 pp, while a 100 ms timing shift costs 4.1 pp (both at 9 m/s, our tested maxima).

What this paper asks. The usual remedy improves the input to the planner — higher-bandwidth sensing, learned prediction, larger training sets — while the objective handed to the optimizer stays a *point target* (Section III). We ask the complementary question: how should the objective itself be built so that hitting plans stay executable under prediction error, and does such robustness survive a constrained execution speed?

The limits of the point-target paradigm. A common formulation casts hitting as a *point-target* problem [1], [2]: a terminal cost penalizes the end-effector for deviating, at a prescribed instant t_{hit} , from a single predicted target, which a prediction error shifts away from the true contact state. Practical alternatives rely on high-bandwidth sensing or learned

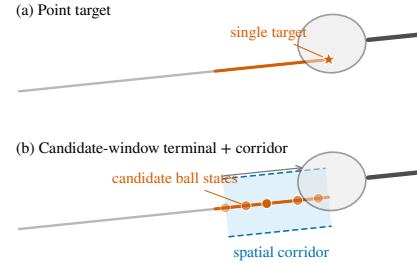


Fig. 1. Top-view schematic of the two cost switches. The spatial corridor penalizes deviations outside the blue band; the candidate-window terminal replaces the single target by candidate states along the trajectory (dots: an illustrative subset, all evaluated at the same terminal state). Panels share coordinates and scale (Figure 4).

robustness, neither of which changes how the objective itself is built.

The gap. A feasible hitting solution is essentially a *spatiotemporal feasible set*: the racket face need only sweep, with an appropriate pose, through a spatial region along the ball trajectory at some candidate instant. The point target compresses this set into a single point. Non-point-target robust planners exist [3], [4], but we are not aware of prior work that attributes temporal and spatial robustness quantitatively under controlled perturbations. We therefore ask: (i) does relaxing the point target improve the hit rate? (ii) are the candidate-set and corridor gains complementary? (iii) does the relaxation survive a constrained execution speed?

Method and contributions. Starting from the point-target terminal of model predictive control (MPC) solved by the iterative linear-quadratic regulator (iLQR), this paper studies the two-switch cost family (6): a *spatial corridor* adds a hinge position-deviation running cost along the predicted ball trajectory, while a *candidate-window softmin terminal* can replace the single-point terminal by a soft aggregation over candidate ball states sampled along that trajectory.

- **C1 (method):** a trajectory-induced candidate-window terminal: softmin over candidate racket states from the predicted trajectory — one shared terminal state, a target set induced by timing uncertainty, not contact-time selection;
- **C2 (method):** a geometric-tolerance corridor regularizer: a running cost from the incoming-ball direction and the racket radius, penalizing only perpendicular deviations outside the corridor;
- **C3 (analysis):** controlled same-seed evaluation (four-

tier ablation, perturbation grid, TCP speed-limit pairing) identifying when each mechanism helps, adds nothing detectable, or hurts.

Artifact. An anonymized artifact containing code, configurations, per-episode results, and reproduction scripts is available at https://anonymous.4open.science/r/Tennis_robot_789741.

II. RELATED WORK

A. Hitting and Throwing Tasks

Recent batting systems use polynomial-primitive planning under 1 kHz visual servoing [5], fixed-horizon MPC, hierarchical learned whole-body control, and motion primitives with MPC and Bayesian optimization [6]–[8], commonly combining point-target objectives with high-bandwidth perception or large training sets; this paper asks the complementary question of how much robustness the cost design itself can buy.

B. Trajectory Optimization and Tube MPC

iLQR/differential dynamic programming (DDP) [1], [2] are the standard solvers for dynamic manipulation; tube MPC theory [9] handles disturbances via state-tube constraints. Our corridor migrates the tube concept to the *task-space path* of hitting, but as a soft cost: we claim neither disturbance invariance nor recursive feasibility. Set-valued terminals [4] extend the DDP terminal to a data-inferred ellipsoidal set; ours adds physically driven explicit structure with a per-term ablative decomposition.

C. Event-Driven Approaches and Contact-Time Uncertainty

The contact instant is a key variable of dynamic manipulation: prior work reparameterizes and optimizes contact time [3] and plans robustly over contact-timing uncertainty sets [10] — requiring precise event detection, accurate models, or hard robustness. Our candidate-window softmin is instead a differentiable *soft* aggregation, free of event switching; ours occupies the candidate-induced, soft-corridor, simulation-only cell.

III. PROBLEM FORMULATION

A. Optimization Problem

Given noisy observations of ball position and velocity, the task is to choose, within the remaining flight time, control commands for the right arm such that the racket face meets the ball with the prescribed pose and speed — the hitting action whose natural completion is a ball returned into the court — while every joint and speed constraint is satisfied. In MPC form, each replan solves the finite-horizon problem

$$\min_{\mathbf{u}_{0:N-1}} \sum_{k=0}^{N-1} \ell_k^{\text{run}}(\mathbf{x}_k, \mathbf{u}_k) + \ell_N^{\text{term}}(\mathbf{x}_N), \quad (1)$$

subject to the discrete dynamics $\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k)$ and the constraint set $\mathcal{C}$, where the running cost ℓ_k^{run} penalizes control effort and smoothness, ℓ_N^{term} encodes the hitting requirement at the predicted contact, and $\mathcal{C}$ collects the joint, TCP speed, and midline constraints of Section III-E. Prediction uncertainty

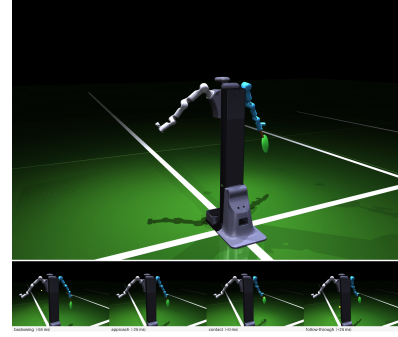


Fig. 2. MuJoCo scene (Section III). Top: RM-65B on its pole 60 ms before contact — right arm (blue) with vertically mounted racket (orange handle, green face, radius 0.12 m), left arm at zero, ball (yellow) inbound. Bottom: backswing, approach, contact, follow-through of one recorded hit.

acts through the objective: the target data of ℓ_N^{term} are built from a noisy ball estimate (Section III-C), and the point-target base with its two switchable relaxations (corridor, candidate window) is defined in Section IV; Section III-D formalizes what counts as a hit.

B. Robot and Dynamics

We consider a dual-arm RM-65B manipulator on a vertical pole; the right arm carries a vertically mounted tennis racket at the flange. The hitting task engages only the right arm, so we model that arm alone: the RM-65B is an integrated dual-arm unit whose left arm cannot be detached from the shared base, but it plays no role in the task and is kept at its zero configuration (it appears in the simulation scene for visual fidelity only). The state collects the joint angles and velocities of the right arm,

$$\mathbf{x} = [\mathbf{q}^\top, \dot{\mathbf{q}}^\top]^\top \in \mathbb{R}^{12}, \quad \mathbf{q} \in \mathbb{R}^6,$$

and the control input is the joint torque $\mathbf{u} \in \mathbb{R}^6$ (torque mode). The scene and one recorded hit are shown in Figure 2.

The discrete dynamics $\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k)$ is provided by MuJoCo; its linearization (batched C++ implementation: closed-form mass matrix and actuator terms, bias-force derivatives by central differences, validated against a finite-difference linearization) supplies the iLQR solver. The end-effector is the racket face center (the `racket_center` site).

C. Ball Prediction and Candidate Hit Windows

The ball is a standard tennis ball (radius 0.033 m, within ITF specifications). From ball position and velocity observations, the planning layer extrapolates a gravity-only parabola $\hat{\mathbf{p}}_b(t)$ (zero spin, no air drag; see limitations). The nominal hit time t_{hit} and hit point $\hat{\mathbf{p}}_{\text{hit}} = \hat{\mathbf{p}}_b(t_{\text{hit}})$ follow from intersecting that trajectory with the racket’s reachable region, and a *candidate hit window* $\mathcal{T} = \{t_{\text{hit}}^{(1)}, \dots, t_{\text{hit}}^{(K)}\}$ of half-width 50 ms is built around t_{hit} , each candidate carrying a desired triple of end-effector position, velocity, and normal.

D. Definitions

Definition 1 (Successful hit): Treating the ball and the racket as a geometry pair, a *contact* occurs if they form a narrow-phase collision pair within an episode. With racket face speed

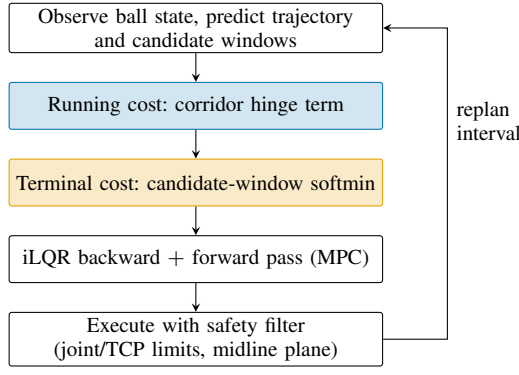


Fig. 3. MPC-iLQR framework. Shaded blocks are the two cost modifications; the point-target baseline (*none*) drops both.

v_r at contact, $v_r > 0.3$ m/s yields an *active hit* and $0 < v_r \leq 0.3$ m/s a *passive contact* (the ball blocks against a nearly stationary racket). The metric throughout this paper is the *active-hit rate* over valid runs; passive contacts are counted separately and never enter it (below 2% of valid runs in every reported cell). A miss is the absence of any contact over the episode.

E. Constraints

Four-fold joint limits (position, velocity, acceleration, torque), a nominal TCP speed cap (1.8 m/s by default, 1.0 m/s for the constraint set inspired by the real robot’s limits; enforced at every step, including the hit neighbourhood), and a body-midline plane wall ($x \geq -0.1$ m) are enforced stage by stage through a multi-layer safety filter (Section IV-D).

IV. METHOD

A. Overall Framework

With the state $\mathbf{x} \in \mathbb{R}^{12}$ and control $\mathbf{u} = \boldsymbol{\tau} \in \mathbb{R}^6$ of Section III, the discrete dynamics $\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k)$, $k = 0, \dots, N-1$, is linearized ($\mathbf{A}_k, \mathbf{B}_k$; batched C++ implementation, see Section III), and the iLQR solver computes gains ($\mathbf{K}_k, \mathbf{k}_k$) by a backward Riccati recursion and updates the trajectory by a forward line search with Levenberg–Marquardt regularization (Figure 3 gives the pipeline overview).

The cost of (1) starts from the point terminal (2) and is modified by two independently switchable mechanisms (6): the running cost can gain the *spatial-corridor term* ℓ_k^{corr} (4), and the terminal cost can be the *candidate-window softmin* ℓ_N^{sm} (5):

$$\ell_N^{pt}(\mathbf{x}_N) = \frac{1}{2} \|\mathbf{p}_{ee} - \mathbf{p}_{hit}\|_{\mathbf{Q}_p}^2 + \frac{1}{2} \|\mathbf{v}_{ee} - \mathbf{v}_{hit}\|_{\mathbf{Q}_v}^2 + \frac{1}{2} Q_n \|\mathbf{n}_{rack} - \mathbf{n}_{des}\|^2 \quad (2)$$

(2) penalizes the racket face center for deviating from a single desired hit point $\mathbf{p}_{hit}$, velocity $\mathbf{v}_{hit}$, and normal $\mathbf{n}_{des}$; all mechanisms below are stated relative to it.

B. Spatial-Corridor Running Cost

The corridor adds a lateral-tolerance running cost: it does not replace the terminal target — which remains the softmin (5) in full and the point terminal (2) in corridor-only — but shapes the approach so that the racket stays close to

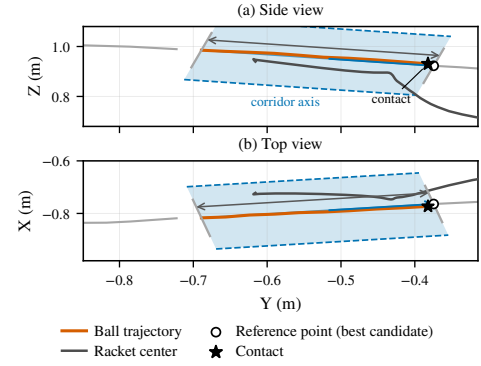


Fig. 4. Spatial corridor along the predicted ball trajectory, zoomed on the candidate window, drawn here as the 50 ms of the trajectory ending at the best candidate step (the full flight is in Figure 1): (a) side and (b) top view of one swing, sharing the forward-axis range. The corridor is a ± 0.12 m band around a straight-line approximation of the ball line within the window (orange: window segment of the trajectory; gray: outside it); the circle is the reference point at the best candidate step, the star the actual contact.

the predicted ball trajectory line within the candidate window. Let $\mathbf{p}_{ball,ref}$ be the ball position at the best candidate step (the corridor axis passes through it) and $\hat{\mathbf{d}}_{ball} = \text{normalize}(\sum_k \bar{w}_k \mathbf{v}_{ball,k})$ the window-averaged trajectory-line direction (weights $\bar{w}_k$ as defined in (5)). The perpendicular projection matrix is

$$\mathbf{P}_\perp = \mathbf{I} - \hat{\mathbf{d}}_{ball} \hat{\mathbf{d}}_{ball}^\top, \quad (3)$$

and the corridor running cost is a hinge loss,

$$\ell_k^{corr}(\mathbf{x}_k) = \frac{1}{2} s_k q_{corr} \left[\max(0, \|\mathbf{P}_\perp(\mathbf{p}_{ee,k} - \mathbf{p}_{ball,ref})\| - r_{racket}) \right]^2, \quad (4)$$

where $r_{racket} = 0.12$ m is the racket radius (the corridor half-width), q_{corr} a scalar weight, s_k a per-step weight scaling (both instantiated below), $\hat{\mathbf{d}}_{ball}$ a window-averaged trajectory direction, and ℓ_k^{corr} is applied only to the steps k inside the candidate window. Only the *perpendicular* deviation is penalized — leading or lagging along the line (timing error) is free, so the solver does not commit to a single time-space correspondence — and inside the band the cost is zero, encoding a contact tolerance of one racket radius around the ball center (Figure 4). The band is a geometric surrogate, not a contact guarantee; alignment is carried by the softmin terminal.

C. Candidate-Window Terminal

The point target also forces “when” to hit to be unique. We center a candidate window $\mathcal{W} = \{k : |t_k - t_{best}| \leq 50 \text{ ms}\}$ at the predicted hit step k_{best} , form a candidate cost c_i for each $i \in \mathcal{W}$ in the form of (2), and aggregate with a softmin:

$$\ell_N^{sm}(\mathbf{x}_N) = -\frac{1}{\beta} \log \sum_{i \in \mathcal{W}} \bar{w}_i \exp(-\beta c_i), \quad \beta = 5.0, \\ \bar{w}_i \propto \exp[-(t_i - t_{best})^2 / (2\sigma^2)], \quad \sum_{i \in \mathcal{W}} \bar{w}_i = 1, \quad (5)$$

with $\sigma = 50$ ms (the normalization contributes only a state-independent constant; the weights favor candidates near the nominal prediction and are not a calibrated probability model of prediction error). Every candidate cost c_i is evaluated at the *same* terminal state $\mathbf{x}_N$ against its own desired triple $(\mathbf{p}_i, \mathbf{v}_i, \mathbf{n}_i)$: candidate i is the target state the racket would have to meet at the terminal instant if the contact-time prediction were off by $t_i - t_{best}$ along the same trajectory. No cross-time state coupling is introduced, so the softmin represents a *target set induced by timing uncertainty* — alternative possible target states at one terminal instant — rather than a contact-time selection mechanism within a single solve. Replanning refreshes the prediction and target set between solves. Each candidate cost is quadratic in a task-space residual, $c_i(\mathbf{x}) = \frac{1}{2} \mathbf{r}_i^\top \mathbf{W}_i \mathbf{r}_i$ with $\mathbf{r}_i = (\mathbf{p}_{ee} - \mathbf{p}_i, \mathbf{v}_{ee} - \mathbf{v}_i, \mathbf{n}_{rack} - \mathbf{n}_i)$ and $\mathbf{W}_i \succeq 0$. ℓ_N^{sm} enters the backward recursion as an ordinary terminal cost, evaluated with the Gauss–Newton Hessian $\sum_i \alpha_i \mathbf{J}_i^\top \mathbf{W}_i \mathbf{J}_i$, where $\mathbf{J}_i = \partial \mathbf{r}_i / \partial \mathbf{x}$ and $\alpha_i = \bar{w}_i e^{-\beta c_i} / \sum_j \bar{w}_j e^{-\beta c_j}$: a weighted sum of positive-semidefinite terms keeps the Riccati recursion well posed in practice, further stabilized by Levenberg–Marquardt damping. Residual-curvature and the softmin-induced negative covariance Hessian terms are omitted in this positive-semidefinite Gauss–Newton approximation. As $\beta \rightarrow \infty$ the aggregation becomes a hard min, and empirically a hard-min terminal ($\beta=10^6$) is indistinguishable from the softmin (Section VI): the operative mechanism is the *candidate target set*, not aggregation smoothness. The two independently switchable terms ((6)) are *designed* to tolerate spatial and temporal prediction errors, respectively; Section VI measures the conditions under which each design intent pays off instead of assuming it.

Reproduction details. At the 5 ms step the ± 50 ms window spans 21 slots; workspace, height, and IK-margin filters retain 2–21 (typically ≈ 13). The corridor scaling s_k equals the normalized weight $\bar{w}_k$; $q_{corr} = 0.01 q_p = 5 \times 10^2$ with $\mathbf{Q}_p = q_p \mathbf{I}$, $q_p = 5 \times 10^4$.

D. MPC Integration and Safety Filtering

Replanning. Replanning occurs every 30 steps (150 ms), scheduled by the steps remaining to the hit: far uses the Jacobian-transpose guess $\mathbf{u} \propto \mathbf{J}_p^\top (\mathbf{p}_{hit} - \mathbf{p}_{ee})$ with zero iLQR iterations, while mid and near add iterations and hard-constraint checks. The control penalty $\mathbf{R}$ anneals as the hit approaches, with an extra decay on joint 1 for a shoulder-driven swing.

Safety filtering. Each step passes three checks: four-fold joint limits, a nominal TCP speed cap v_{tcp}^{max} , and the body-midline plane wall ($x \geq -0.1$ m, pushed back by PD when violated). The cap is enforced at every step, including the hit neighbourhood: a fallback gain sequence $\alpha_s \in \{1.0, 0.8, 0.6, 0.4, 0.2, 0.0\}$ scales the candidate control, and the largest value whose *one-step prediction* passes every check is kept (scaling, not clipping). The realized peak speed can overshoot the cap transiently (Section VI); if every scaling fails, a damping brake ($\mathbf{u} = -20 \dot{\mathbf{q}}$) applies as an emergency

fallback, flagged as a fallback rather than a checked-safe control; it fired in 3 of 40 recorded real-time runs.

Follow-through. For 60 steps (300 ms) after contact the end-effector keeps accelerating opposite to the incoming ball (length 0.5 m), preventing deceleration from shifting the hit point.

Real-time budget. Upon serve detection the controller performs one synchronous 30-iteration first plan before motion begins, with simulated time frozen — initialization, not offline precomputation, since it already consumes the serve’s prediction (Section VI quantifies the consequence, including unfrozen end-to-end runs). Steady-state replanning fits the 150 ms budget, and asynchronous execution prevents solver latency from blocking the control loop.

E. Point-Target Baseline (Degenerate Case)

One family, two switches. With ℓ_k^{base} the control-effort and smoothness running cost, the family reads

$$J_{a,b} = \sum_k (\ell_k^{base} + a \ell_k^{corr}) + (1-b) \ell_N^{pt} + b \ell_N^{sm}, \quad a, b \in \{0, 1\}, \quad (6)$$

so the corridor *adds* a running term while the softmin *replaces* the point terminal. The four ablation tiers are $\text{full } (a,b) = (1,1)$, *corridor-only* $(1,0)$, *candidate-set-only* $(0,1)$, and *point-target* $(0,0)$. Apart from these two switches all tiers share identical weights, warm-start, initialization, and safety rules; the sensitivity sweep over β , the window half-width, and r_{racket} is in Section VI.

V. EXPERIMENTS

Common protocol. All experiments run the dual-arm RM-65B in MuJoCo (right arm, 6 driven joints; left arm zeroed) at a 5 ms step, with serves drawn from a randomized serve box (ball speed 7–15 m/s). The metric is the active-hit rate of Definition 1 (Section III-D): a ball–racket collision pair with racket face speed > 0.3 m/s. For all multi-tier comparisons, seeds $(1 \dots N)$ are paired across tiers, isolating the algorithmic configuration under otherwise identical conditions. Perturbations are injected on the planning side at each replan. The temporal perturbation shifts the predicted hit step, moving the horizon, the iteration-phase schedule, and the control-penalty annealing identically across tiers. The spatial perturbation offsets the predicted hit point perpendicular to the predicted trajectory, shifting the point-target terminal and, in every tier, the Jacobian-transpose warm start, whereas the candidate window and the corridor axis are rebuilt from the current ball observation and do not carry it; it thus measures a biased single point target while the trajectory geometry is still available (Section VI reports the end-to-end variant). Both magnitudes are uniform in $[0, \max]$ with a random, seeded sign, decaying with the replan index, emulating a prediction that improves as the ball approaches. Runs without a verdict are excluded (0.9–3.0% of rows, evenly distributed across tiers): process-level failures (timeout or crash); solver-level failures fall back to Jacobian-transpose control and still produce verdicts. Physics integrates at the 5 ms step without substepping, with the ball–racket collision pair enabled only in the hit neighbourhood

TABLE I

GAIN (PP) OVER THE POINT-TARGET BASELINE UNDER THE TWO TCP CAPS (7 M/S, $n = 394$ PAIRED SEEDS); PAIRED MCNEMAR p ; DiD = PAIRED PER-SEED GAIN DIFFERENCE (BOOTSTRAP 95% CI; SIGN TEST ON DIRECTION); p -VALUES UNADJUSTED, PRE-SPECIFIED COMPARISONS.

Config.	Gain over point-target (pp)		p	
	TCP 1.8	TCP 1.0	TCP 1.8	TCP 1.0
corridor-only	+3.3	+5.8	0.171	0.027
candidate-set-only	+1.3	-11.9	0.668	< 0.001
full	+1.5	-12.7	0.594	< 0.001
Paired DiD (gain shift from TCP 1.8 to TCP 1.0):				
corridor-only	+2.5 pp (CI [-4.1, 9.1]; sign test $p = 0.466$)			
candidate-set-only	-13.2 pp (CI [-19.8, -6.6]; sign test $p < 0.001$)			
full	-14.2 pp (CI [-20.8, -7.6]; sign test $p < 0.001$)			

(within 30 steps and 0.35 m of the nominal contact); the racket face is a thick ellipsoid (half-thickness 2.5 cm). Because the design is paired, all comparisons use the *exact paired McNemar test* on same-seed outcomes with paired-bootstrap 95% confidence intervals; the limit $\times$ mechanism interaction uses a paired difference-in-differences (per-seed gain difference); grid heatmaps are corrected by the Holm procedure over the 20 cells.

A. Capability Sweep and the Real-Robot Constraint Set

The speed sweep (9 speeds $\times$ 200 seeds, default TCP limit 1.8 m/s) maps the task’s capability envelope; the constraint-set comparison (7 m/s $\times$ 200 seeds, same seeds under both limit layers; the tight one a nominal 1.0 m/s cap enforced at every step, Section IV-D) prices that set (Figure 6, Figure 7).

B. Four-Tier Ablation and the Perturbation Grid

We refer to the four ablation flags *full*, *tube_only*, *softmin_only*, *none* as *full*, *corridor-only*, *candidate-set-only* (softmin implementation), and *point-target*.

Nominal four-tier ($n \approx 98$ per cell) runs all tiers at 7/9/12 m/s without perturbation. **Perturbation grid** (9 m/s, replicated at 7 m/s) crosses time maxima $t \in \{0, 10, 25, 50, 100\}$ ms with spatial maxima $s \in \{0, 0.05, 0.1, 0.2\}$ m, 20 cells per tier ($n \approx 1960$). **Pre-specified high-power corners** (9 and 12 m/s, $s \in \{0.2, 0.3, 0.4\}$ m at $t = 0$, plus $t = 50$ ms at $s = 0.2$ m, $n = 1481$ –2986 per cell) targets the corridor’s *independent* value with sufficient power; comparisons were fixed before the runs. Per-condition rates of the four tiers are collected in Table II; results are reported per research question in Section VI.

C. Limit $\times$ Mechanism Pairing

The limit pairing (7 m/s, $2 \times 4 \times 400$ seeds, same-seed aligned across both limit layers and all tiers) crosses the two TCP layers with the four tiers to test whether the mechanism ranking depends on the speed budget (Table I); representative same-seed diagnostics sanity-check the hit/miss separation (Section VI).

D. Real-Time Budget

The real-time study runs 30 synchronous and 10 asynchronous episodes at 9 m/s on an idle machine (single workstation, one solver thread; Intel i9-12900K), logging every replan duration and per-step latency (Figure 8).

E. Additional Checks

Five targeted checks close the identified gaps. (i) *End-to-end unfrozen runs*: the first plan no longer freezes simulated time, so the ball keeps flying during the warm-up (7/9/12 m/s $\times$ 4 tiers $\times$ 200 seeds, plus a 5-iteration variant at 9/12 m/s). (ii) *Shared-input bias*: one perturbed ball state feeds every planning module (hit point, window, corridor axis, timing, reachability; 9 m/s, five (t, s) cells, 100 seeds per tier; Table III). (iii) *Selection baseline*: the most reachable candidate from the window (largest IK joint margin) replaces the softmin terminal in a point-target MPC run (7/9/12 m/s $\times$ 200 seeds). (iv) *Speed $\times$ cap grid*: $\{7, 9\}$ m/s crossed with TCP $\{1.0, 1.4, 1.8\}$ m/s, 4 tiers, 100 seeds per cell (Figure 7(c)). (v) *Hit quality*: nominal reruns at 9/12 m/s record the post-impact ball velocity (100 seeds per tier).

VI. RESULTS AND DISCUSSION

A. RQ0: Capability Envelope and the Cost of the TCP Constraint Set

Under default limits the active-hit rate over $7 \rightarrow 15$ m/s peaks at 8–9 m/s ($\sim 94\%$) and falls to 58.0% at 15 m/s ($n \approx 198$; Figure 7(a)). The robot-derived constraint set (nominal TCP cap 1.0 m/s, enforced at every step) at 7 m/s lowers the rate from 84.4 to 35.9% (-48.7 pp, paired CI $[-56.9, -40.6]$, $p < 0.001$) and compresses the mean racket speed on hit rows from 1.50 to 0.88 m/s: an execution-time limit, not a planning guideline. We call these settings *speed-rich* and *speed-scarce*.

B. RQ1: What Does the Candidate-Window Terminal Contribute?

Under nominal conditions the candidate window captures nearly the entire gain over the point target: +17.4 pp at 9 m/s (CI $[+9.2, +25.5]$) and +32.7 pp at 12 m/s (CI $[+22.4, +42.9]$), with full = candidate-set-only there (0.0 pp, $p = 1.0$; Figure 5(a)). At 7 m/s the gain is +2.1 pp with no pairwise difference detected (all $p \geq 0.40$; the TCP 1.8 layer agrees, Table I). On the merged 20-cell grid candidate-set-only reaches 82.8% against 73.9% for the point target (+8.9 pp, CI $[+6.9, +11.0]$), with full and candidate-set-only differing by 0.3 pp ($p = 0.06$); after Holm adjustment only the two zero-spatial-error cells are individually significant (Figure 5(b)). A per-seed interaction $I(t, s) = [H_{sm}(t, s) - H_{pt}(t, s)] - [H_{sm}(0, 0) - H_{pt}(0, 0)]$ is negative in all 19 perturbed cells, from -2.0 to -19.4 pp (mean -8.9 pp): the advantage is largest unperturbed and *narrows* under both perturbation types, so the nominal gain partially persists under perturbation rather than splitting by mechanism.

Selection baseline and hit quality. Two checks guard the headline metric. First, must the set be embedded, or does selecting one candidate suffice? A selection baseline picks

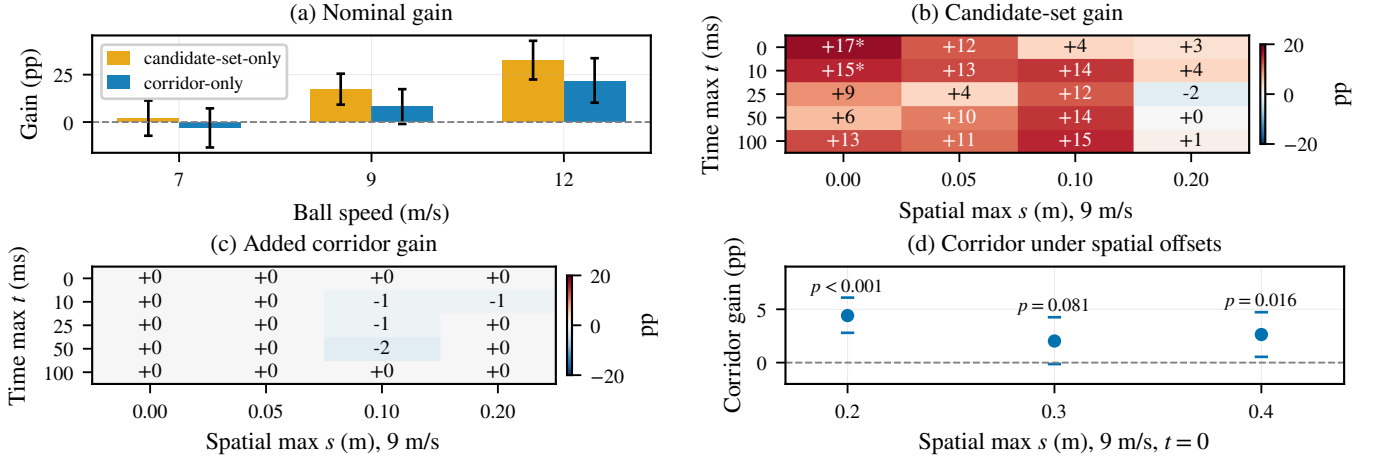


Fig. 5. Mechanism attribution: the candidate window carries nearly the whole nominal gain; corridor-only is significant under selected spatial perturbations; once the window is active no further corridor effect is detected. (a) Nominal gain vs. ball speed (bootstrap 95% CI). (b) candidate-set-only and (c) added corridor gain on the 20-cell grid at 9 m/s, shared ± 20 pp scale; stars: Holm-significant. (d) corridor-only value vs. magnitude at 9 m/s, $t=0$ (pre-specified corners; uncorrected p). The spatial perturbation biases the point target only; the end-to-end variant (Table III) biases the ball state consumed by every module.

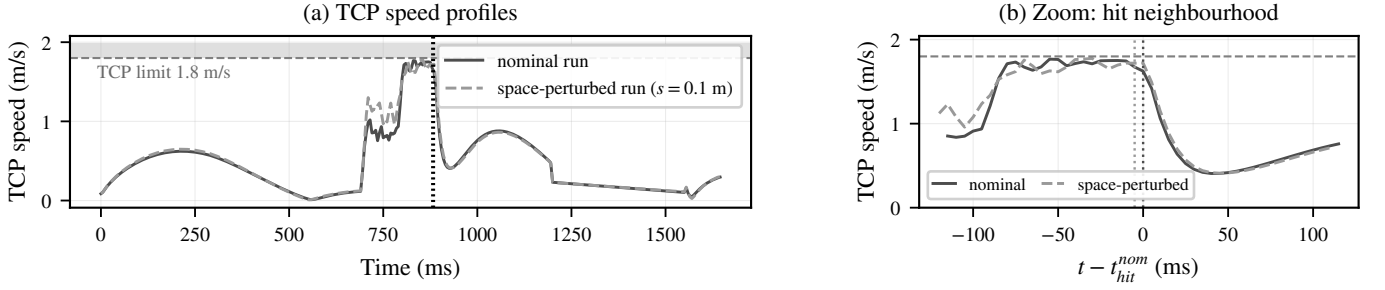


Fig. 6. Representative swing under the default limits: (a) TCP speed profiles of a nominal and a space-perturbed run ($s = 0.1$ m) against the 1.8 m/s cap; (b) hit-neighbourhood zoom: the runs differ by up to $\Delta v_{max} = 0.38$ m/s and -5 ms at contact (dotted lines matched to each run's line style).

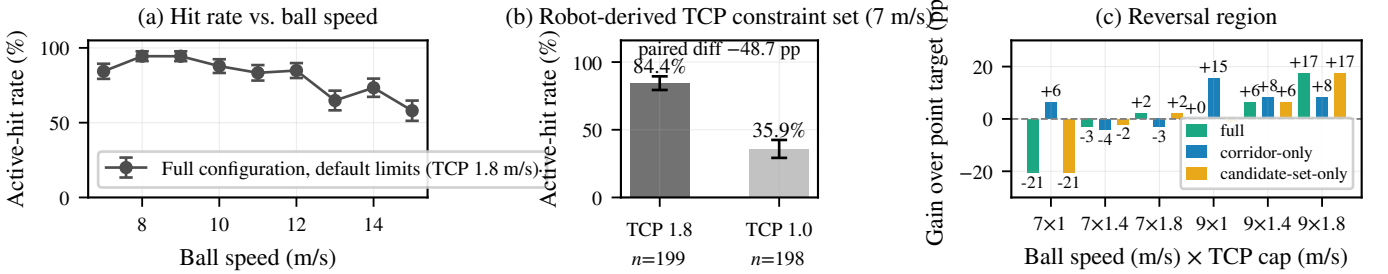


Fig. 7. Capability and the cost of the TCP constraint: (a) active-hit rate vs. ball speed under default limits; (b) default vs. the 1.0 m/s constraint set at 7 m/s; Wilson 95% intervals and per-condition n , with the paired difference over the $n=197$ same-seed pairs. (c) Reversal region: paired gain (pp) over the point target on the speed $\times$ cap grid; the loss is localized to the 7 m/s $\times$ 1.0 m/s cell.

the single most reachable candidate (largest IK joint margin) and runs the point-target MPC on it: 57.4/62.4/44.4% at 7/9/12 m/s versus 83.5/95.9/85.7% for the soft-aggregated terminal (paired +27.8/ +29.6/ +39.8 pp, all $p < 10^{-4}$); at 7 m/s the point target itself beats the selection by 25.8 pp ($p < 0.001$). The set must be embedded in the objective, not merely searched. Second, active contact yields effective returns, not grazes: on nominal reruns, median outgoing speeds are 7.6 m/s (9 m/s incoming) and 9.6 m/s (12 m/s), and 88–98% of window-tier hits return the ball across the net ($v_y < 0$) versus 66–79% without it.

C. RQ2: When Does the Corridor Provide Independent Value?

Even without injected error, the corridor improves over the point target where time margins are tight: +21.4 pp at 12 m/s (CI [+10.2, +33.7], $p=0.001$, $n=98$) and +8.2 pp at 9 m/s ($p=0.12$, n.s.), with -3.1 pp at 7 m/s (n.s.) — its standalone value is a time-margin effect, not one confined to spatial-error conditions. Under a biased point target at the pre-specified corners the independent value is +4.4 pp (9 m/s, $t=0$, $s=0.2$ m; CI [+2.8, +6.1], $p < 0.001$, $n=2970$) and +6.9 pp (12 m/s; CI [+5.4, +8.4], $p < 0.001$, $n=2986$); at 9 m/s the estimate stays positive at $s=0.3/0.4$ m (+2.0/+2.6 pp,

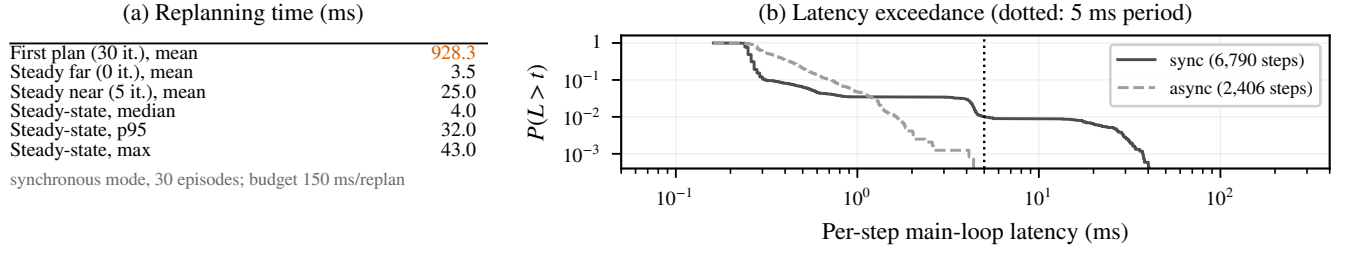


Fig. 8. Real-time budget after initialization: steady-state replanning meets the 150 ms budget with ample margin, and the per-step latency tail of the synchronous mode is what causes main-loop stalls. (a) Replanning time per phase (numeric summary, synchronous mode; the first plan consumes the serve’s prediction before any motion). (b) Exceedance probability $P(L > t)$ of the per-step main-loop latency, log axes.

TABLE II

ACTIVE-HIT RATE (%) OF THE FOUR CONFIGURATIONS ACROSS CONDITIONS; n = VALID RUNS PER CONFIGURATION, GIVEN AS THE RANGE ACROSS THE FOUR (THE PERTURBATION-GRID ROW POOLS ITS 20 CELLS); EXACT INTERVALS ARE IN THE ARTIFACT.

Condition	full	corridor-only	candidate-set-only	point-target	n	Design
Nominal 7 m/s	83.8	78.4	83.5	81.4	97–99	nominal four-tier
Nominal 9 m/s	95.9	86.7	95.9	78.6	98	nominal four-tier
Nominal 12 m/s	85.7	74.5	85.7	53.1	98	nominal four-tier
Perturbed grid (20 cells, 9 m/s)	82.6	81.2	82.8	73.9	1960–1962	perturbation grid
Corner 12 m/s, $t=0$, $s=0.2$ m	64.8	61.6	64.8	54.7	2986–2987	high-power corners

n.s./raw $p=0.016$), and a 50 ms temporal shift leaves +2.0 pp ($p=0.115$). In the full system this independent value is covered by the candidate window: at the corners full and candidate-set-only differ by ± 0.0 pp ($p = 1.0$), and on the grid no cell of full – candidate-set-only is significant even without correction (Figure 5(c)) – an absence of a detected effect, not an equivalence claim. The two mechanisms therefore *substitute* rather than complement under these conditions.

Shared-input check. Above, only the point target carries the spatial bias. In the end-to-end variant, one perturbed ball state feeds every planning module — hit point, candidate window, corridor axis, timing, and reachability — so the bias is uniform (Table III). The corridor retains independent value in the spatial channel: +11.2 pp at (0, 0.1) (CI [+4.1, +19.4], $p=0.013$) and +9.2 pp at (0, 0.3) (CI [+3.1, +16.3], $p=0.012$), with no significant effect in the timing cells; the candidate window adds +5.1 pp (n.s.) and full equals candidate-set-only in every cell (0 discordant pairs). Under a uniform bias the corridor’s value therefore survives independently, while the window’s nominal advantage narrows.

Sensitivity of the nominal parameters. The nominal choices are not knife-edge: at 9 m/s (200 seeds per setting), full and candidate-set-only stay within 93.9–94.9% across $\beta \in \{1, 10\}$, half-widths {25, 75} ms, and radii {0.08, 0.16} m, and corridor-only at 84.8% for both radii ($p = 0.18$). A numerical hard-min over the same set ($\beta=10^6$) agrees with the nominal softmin on all 98 shared seeds (zero discordant pairs): the gain arises from the candidate set itself, not soft aggregation.

D. RQ3: How Does the TCP Constraint Change the Ranking?

The limit $\times$ mechanism pairing changes the tier ranking (Table I): under TCP 1.8 no tier separates from the point target (all $p \geq 0.17$), while under the nominal 1.0 m/s

cap candidate-set-only loses -11.9 pp and full -12.7 pp (paired $p < 0.001$; mean paired DiD $-13.2/-14.2$ pp, CI $[-19.8, -6.6]/[-20.8, -7.6]$), whereas corridor-only gains +5.8 pp (CI [+1.0, +10.7], $p = 0.027$), the highest tier under the limit. On the $n = 93$ seeds where both tiers hit, the point target attains the smallest mean hit-time error (14.4 ms, 0.76 m/s swing) vs. 17.2 ms (0.87 m/s) for the softmin, consistent with the softmin spending part of its speed budget hedging across candidates; a numerical hard-min under the same cap moves the rates by at most 0.3 pp (CI $[-0.8, 0.0]$, one discordant pair each), so the smoothness of the aggregation alone does not explain the decline. Peak TCP speeds exceed the cap in 19/8/19/6% of episodes (tiers in order), mostly marginal and transient ($p90 \leq 1.01$ m/s); the 13–14 per tier with larger peaks (> 1.2 m/s) are all misses at a shared unstable initial condition, not a mechanism difference. To locate the reversal rather than assume it, we crossed ball speed with the cap (Figure 7(c)): the loss is localized to the 7 m/s $\times$ 1.0 m/s cell (full and candidate-set-only both -20.6 pp), already -2 to -4 pp at the 1.4 m/s cap, and at 9 m/s the window is harmless-to-beneficial under every cap while the corridor adds +15.3 pp even at 1.0 m/s. The reversal is thus a property of the most speed-scarce cell, not a general failure mode of the objective family.

E. RQ4: Does Replanning Meet the Runtime Budget?

Steady-state replanning takes 4 ms median (32 ms p95, 43 ms max; 291 replans), inside the 150 ms budget. The bottleneck is the first plan (a synchronous 30-iteration solve, 928.3 ms), which consumes the serve’s prediction before motion starts with simulated time frozen: the steady-state budget describes the loop *after* initialization, not the response to a mid-run serve. The control-relevant tail (Figure 8(b)): the synchronous mode exceeds the 5 ms period on 1.0% of steps,

TABLE III

SHARED-INPUT CHECK, END TO END: EVERY PLANNING MODULE (HIT POINT, CANDIDATE WINDOW, CORRIDOR AXIS, TIMING, REACHABILITY) CONSUMES THE SAME PERTURBED BALL STATE. ACTIVE-HIT RATES (%) AT 9 M/S OVER FIVE (t, s) CELLS, $n=98$ PAIRED EPISODES PER TIER (2,000 TOTAL). THE CORRIDOR GAINS OVER THE POINT TARGET ARE SIGNIFICANT IN THE SPATIAL CHANNEL ($p=0.013/0.012$ AT $(0, 0.1)/(0, 0.3)$); THE CANDIDATE WINDOW ADDS $+5.1$ PP (N.S.), AND FULL EQUALS CANDIDATE-SET-ONLY IN EVERY CELL (0 DISCORDANT PAIRS).

(t, s) cell	full	corridor-only	candidate-set-only	point-target
(0, 0.1)	59.2	65.3	59.2	54.1
(0, 0.2)	32.7	37.8	32.7	32.7
(0, 0.3)	23.5	27.6	23.5	18.4
(25, 0.2)	39.8	32.7	39.8	33.7
(50, 0.2)	36.7	31.6	36.7	37.8

the asynchronous mode on 0.04% — asynchronous replanning buys non-blocking execution, not faster solving.

Unfrozen end-to-end runs. Removing the freeze makes the default first plan infeasible end to end: at 7/9/12 m/s the 30-iteration warm-up (999 ms median) exceeds the serve’s flight time (~ 1.1 s at 7 m/s), and no tier hits in 2,368 episodes. Capping the first plan at 5 iterations (224 ms median) recovers 74.5% ($n=784$, 9/12 m/s), with the tier ordering intact: the candidate window gains $+17.3$ pp at 9 m/s ($p=0.0023$) and $+20.4$ pp at 12 m/s ($p=0.0037$) over the point target. End-to-end deployment therefore needs a sub-300 ms first plan, met at reduced warm-up quality.

F. Discussion

Positioning and implication. Relative to the “point-target + high-bandwidth perception” route [6]–[8] this work is complementary (where sensing is tighter, cost design compensates), and relative to event-time reparameterization [3] and robust contact-timing optimization [10] it delimits the *regime of applicability* of each layer: a robustness layer’s value can change sign; mapping its regime should be standard practice in cost design.

Limitations. The study is simulation-based, isolating the algorithmic factors; no hardware validation is claimed. Prediction error is injected as a hit-point position offset and a timing shift; zero spin and no air drag are modeled. The nominal weights are one configuration, and the one-at-a-time parameter sweep (Section VI) makes the reported boundaries conditional on them. The nominal rates assume the first plan consumes the serve prediction before motion (frozen clock); unfrozen runs need a reduced first-plan budget (Section VI). Evidence covers one task and one arm type; the reversal, though mapped across the speed $\times$ cap grid (Figure 7(c)), is sharpest in the most speed-scarce cell (7 m/s $\times$ TCP 1.0). Active contact is the task metric; post-impact outgoing speed and direction are measured, but net clearance and landing accuracy are not evaluated. Accounting: 123,810 episodes across the reported conditions (the perception sweeps are in the anonymized artifact); 1.3% ended without a verdict, with the exclusion share within 0.5 pp across tiers in every experiment. Every rate, interval, and test statistic here is regenerated by one analysis script from the per-episode logs.

VII. CONCLUSION

The point-target terminal is brittle to spatiotemporal prediction errors because it compresses a feasible time–space set

into a single target. Around an MPC-iLQR instance of it we studied a two-switch cost family — a *spatial-corridor* running cost and a *candidate-window softmin* terminal — over 123k controlled episodes. The candidate window carries the dominant nominal gain, and the set must be embedded rather than searched: selecting the best single candidate loses 27–40 pp. The corridor’s independent value appears where time margins are tight ($+21.4$ pp at 12 m/s, $p=0.001$; n.s. at 9 m/s), survives an end-to-end shared-input bias in the spatial channel ($+11.2$ pp, $p=0.013$), and is subsumed wherever the window is active; under a nominal 1.0 m/s TCP cap the window turns harmful, but the reversal is localized to the most speed-scarce cell (7 m/s $\times$ 1.0 m/s); unfrozen end-to-end runs retain the tier ordering under a reduced first-plan budget. For dynamic manipulation, a cost layer’s robustness is task-condition-dependent and should be mapped, not assumed. Future work: real-robot deployment, continuous rallying, closed-loop perception, and limit-aware mechanism switching.

REFERENCES

- [1] E. Todorov and W. Li, “A generalized iterative LQG method for locally-optimal feedback control of constrained nonlinear stochastic systems,” in *Proc. American Control Conference (ACC)*, Jun. 2005, pp. 300–306.
- [2] Y. Tassa, T. Erez, and E. Todorov, “Synthesis and stabilization of complex behaviors through online trajectory optimization,” in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, Oct. 2012, pp. 4906–4913.
- [3] T. Gossard, T. Köpf, and A. Ziegler, “Event-time hybrid optimal control for robotic table tennis serves,” 2026, arXiv:2608.08157.
- [4] S. Eom and G. Park, “Differential dynamic programming for the optimal control problem with an ellipsoidal target set and its statistical inference,” in *Proc. Int. Conf. Control, Automation and Systems (ICCAS)*, Oct. 2025, pp. 1960–1965.
- [5] T. Senoo, A. Namiki, and M. Ishikawa, “High-speed batting using a multi-jointed manipulator,” in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2004, pp. 1191–1196.
- [6] D. Nguyen, K. D. Cancio, and S. Kim, “High speed robotic table tennis swinging using lightweight hardware with model predictive control,” in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, May 2025, pp. 15 278–15 284.
- [7] Z. Su, B. Zhang, N. Rahmadian, Y. Gao, Q. Liao, C. Regan, K. Sreenath, and S. S. Sastry, “HITTER: A Humanoid Table Tennis robot via hierarchical planning and learning,” 2025, arXiv:2508.21043.
- [8] G. Torrente, G. J. Maeda, D. Grover, M. Tsukamoto, H. Sahloul, and P. Dürr, “Ace! motion planning of professional-level table tennis serves with a robot arm,” 2026, arXiv:2607.06989.
- [9] D. Q. Mayne, J. B. Rawlings, C. V. Rao, and P. O. M. Scokaert, “Constrained model predictive control: Stability and optimality,” *Automatica*, vol. 36, no. 6, pp. 789–814, 2000.
- [10] H. Zhao and M. Khadiv, “Trajectory optimization under contact timing uncertainties,” 2024, arXiv:2407.11478.